

Predicting History of Mental Illness

A Machine Learning Analysis of Lifestyle & Socio-Economic Factors

Project Handover Report

Prepared by: Shaqran

Platform: Databricks (PySpark, Pandas, Scikit-learn, XGBoost)

Dataset source: Kaggle — Depression Dataset (anthonytherrien)

September 2026

1. Introduction

This report documents the end-to-end analysis of a publicly available depression-related dataset, undertaken to identify which lifestyle and socio-economic factors are associated with a recorded history of mental illness, and to build a predictive model capable of estimating that outcome for a new individual based on those factors.

The project was carried out entirely on Databricks, using a combination of PySpark for data ingestion and initial processing, and the Python data science stack (pandas, seaborn, matplotlib, scikit-learn, and XGBoost) for exploratory analysis and modelling. The purpose of this document is to hand over the full body of work — data preparation, exploratory findings, modelling approach, results, and an interactive prediction tool — to a business audience, with enough explanation that a non-technical stakeholder can understand what was done, why it was done that way, and what can and cannot be concluded from the results.

The report is organised to mirror the actual workflow followed during the project: from raw dataset ingestion, through cleaning and exploratory analysis, into model development and tuning, and finally to the results, an interactive prediction widget, and a candid discussion of the model's limitations.

2. Dataset

The dataset used for this project is the "Depression Dataset" published on Kaggle by contributor anthonytherrien, described as a comprehensive collection of health, lifestyle, and socio-economic records intended for the analysis of factors associated with mental illness.

2.1 Size and Structure

- Total records: 413,768 individuals
- Total columns: 16
- Duplicate rows found: 0 (row count was identical before and after de-duplication)
- Missing values: none detected across any column

2.2 Column Overview

Column	Type	Description
Name	Text	Unique identifier for the individual (dropped before modelling)
Age	Numeric	Age in years (range 18–80)
Marital Status	Category	Married / Single / Divorced / Widowed
Education Level	Category	High School through PhD
Number of Children	Numeric	Count of children (range 0–4)
Smoking Status	Category	Non-smoker / Former / Current
Physical Activity Level	Category	Low / Moderate / High
Employment Status	Category	Employed / Unemployed
Income	Numeric	Annual income in dollars (range \$0.41–\$99,999.75)

Column	Type	Description
Alcohol Consumption	Category	Low / Moderate / High
Dietary Habits	Category	Unhealthy / Moderate / Healthy
Sleep Patterns	Category	Poor / Fair / Good
History of Mental Illness	Category (target)	Yes / No — the outcome variable used for prediction
History of Substance Abuse	Category	Yes / No
Family History of Depression	Category	Yes / No
Chronic Medical Conditions	Category	Yes / No

2.3 Choice of Target Variable

The dataset does not contain a column labelled specifically as "depression". The most relevant candidate columns are History of Mental Illness and Family History of Depression. Depression is clinically classified as a mood disorder and falls under the broader category of mental illness, so History of Mental Illness was selected as the prediction target, since it is the only outcome-style column that directly labels the individual's own status (rather than a family history). This is an important scoping decision: the model built in this project predicts general history of mental illness, not a depression-specific diagnosis, and this distinction is reported transparently rather than implied.

2.4 Class Balance of the Target

```
+-----+
|History of Mental Illness| count|
+-----+
|                          No|287943|
|                          Yes|125825|
+-----+
```

Approximately 30% of records are labelled "Yes" (history of mental illness) and 70% "No". This is a moderate class imbalance, which was accounted for by using stratified train/test splitting and by evaluating models on the F1-score (rather than plain accuracy), since accuracy alone can be misleading on imbalanced classification problems.

3. Procedure

This section documents the technical workflow followed, from raw file to a modelling-ready dataset. Screenshots of representative code are included so the process can be reproduced or audited by another analyst.

3.1 Data Ingestion (Databricks / Unity Catalog)

The raw CSV file was uploaded into a Unity Catalog Volume within the Databricks workspace (workspace.default.depre), which keeps the raw file, the cleaned data, and all downstream tables within a single governed location (a Databricks "lakehouse" — avoiding the need to copy data between a separate data lake and a separate analytics warehouse). The file was then loaded into a Spark DataFrame for initial processing:

```
df = spark.read.csv('/Volumes/workspace/default/depre/depression_data.csv',
                    header=True)
display(df)
```

Code: loading the raw CSV from the Unity Catalog Volume into a Spark DataFrame

3.2 Data Type Correction

On initial load, every column — including numeric ones — was read in as a string type. Age, Number of Children, and Income were cast to their correct numeric types (integer, integer, and double respectively) to allow numeric operations, filtering, and later modelling:

```
from pyspark.sql.functions import col
from pyspark.sql.types import IntegerType, DoubleType

df = df.withColumn("Age", col("Age").cast(IntegerType())) \
        .withColumn("Number of Children", col("Number of Children").cast(IntegerType())) \
        .withColumn("Income", col("Income").cast(DoubleType()))
```

3.3 Data Quality Checks

- Row count verified consistent before and after de-duplication (413,768 both times) — no duplicate records.
- Null counts checked across all 16 columns — zero missing values found.
- All 12 categorical columns checked for inconsistent labelling (typos, casing) — none found.
- Numeric ranges checked for impossible values (negative age, negative income, negative children) — none found.

```
df.select("Age", "Income", "Number of Children").describe().show()

df.filter(col("Income").cast("double") < 0).show()
df.filter((col("Age").cast("int") < 0) | (col("Age").cast("int") > 110)).show()
df.filter(col("Number of Children").cast("int") < 0).show()
```

	Age	Income	Number of Children
count	413768	413768	413768
mean	49.00071295991957	50661.707971302756	1.2989718876278493
stddev	18.158759370459084	40624.10056474354	1.2370539679549237
min	18	0.41	0

	max	80	99999.75	4
+	-----+	-----+	-----+	-----+

All three filters for impossible values returned zero rows, confirming the numeric fields required no correction. Age (18–80) and Number of Children (0–4) also fall within sensible real-world bounds.

3.4 Conversion to Pandas for Analysis

Once cleaned, the dataset (a manageable ~414K rows) was converted from a distributed Spark DataFrame into a single in-memory pandas DataFrame, which is simpler and faster to work with for exploratory analysis, visualisation, and scikit-learn modelling at this scale:

```
pdf = df.toPandas()
```

4. Insights (Exploratory Data Analysis)

This section presents the key visual and statistical findings from exploring the cleaned dataset, before any predictive modelling was attempted.

4.1 Distribution of Numeric Fields

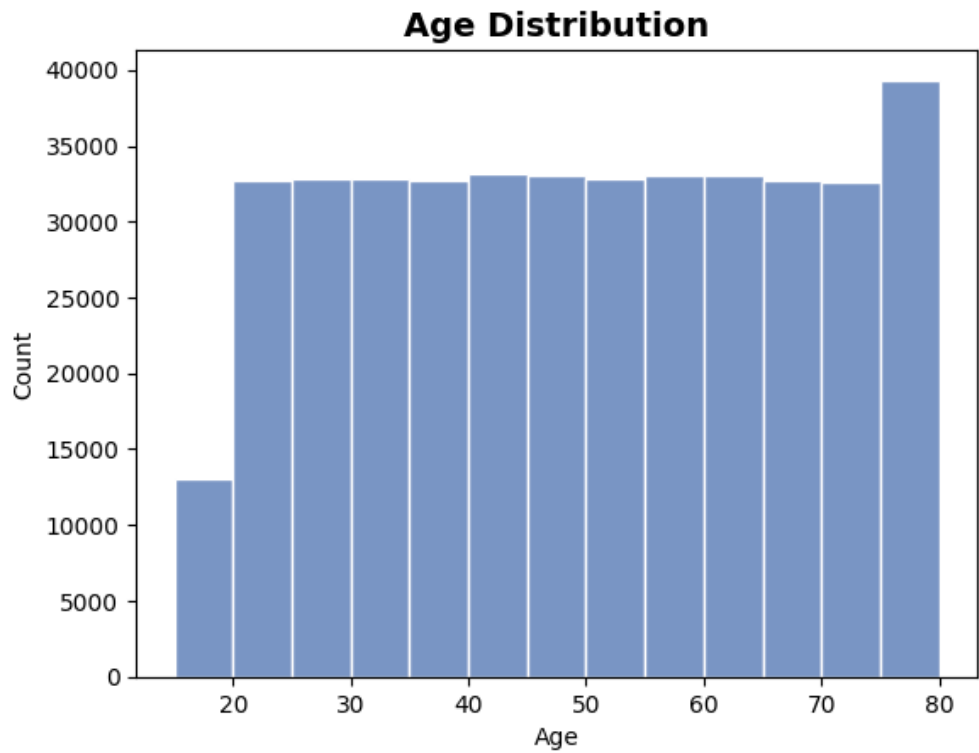


Figure 1: Age distribution — approximately uniform across 18–80, suggesting this field may have been synthetically generated rather than sampled from a real population.

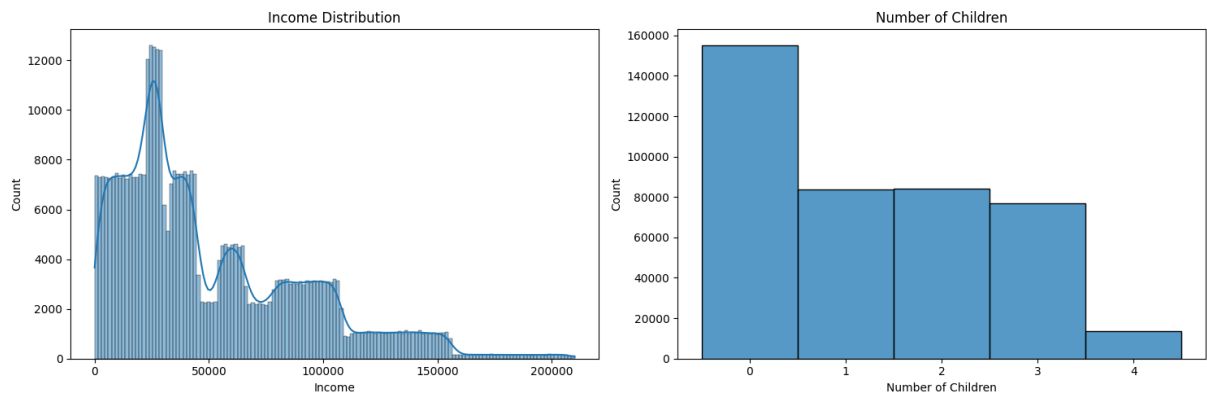


Figure 2: Income distribution (left) shows several overlapping peaks, consistent with income being generated conditionally on another factor such as employment. Number of Children (right) is concentrated at 0–3, with fewer records at 4.

4.2 Categorical Factors vs. History of Mental Illness

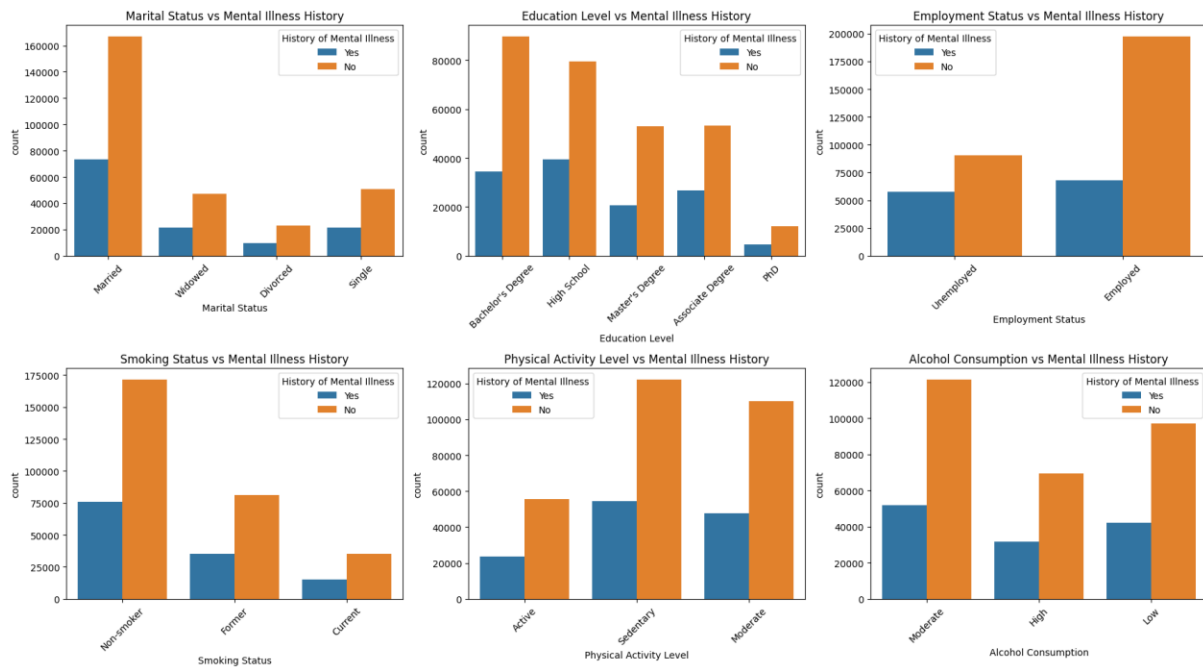


Figure 3: Raw counts of Yes/No history of mental illness across six lifestyle and demographic factors. Because group sizes differ substantially (e.g. far more "Married" than "Divorced" individuals), these raw counts must be read as proportions, not absolute risk — see Section 4.4.

4.3 Age, Income and Mental Illness History

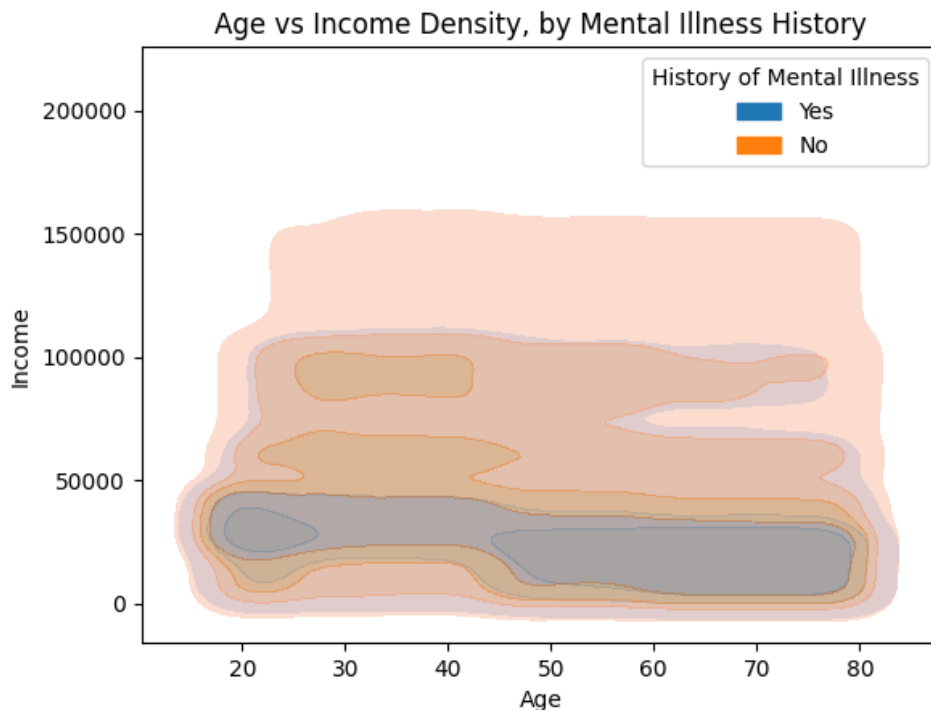


Figure 4: Density of Age vs. Income, split by mental illness history. The heavy overlap between the "Yes" and "No" groups indicates these two factors alone do not clearly separate the two outcomes.

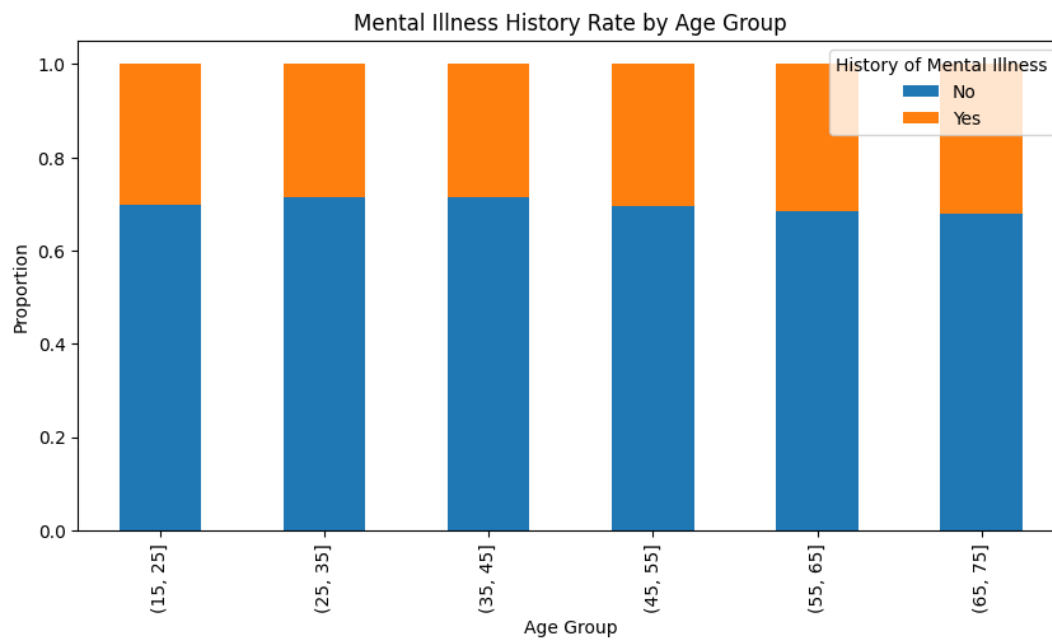


Figure 5: Proportion (not count) of individuals with a history of mental illness, by age bracket. The proportion stays close to ~28–32% across every age group, indicating age has little influence on this outcome on its own.

4.4 Income by Employment Status

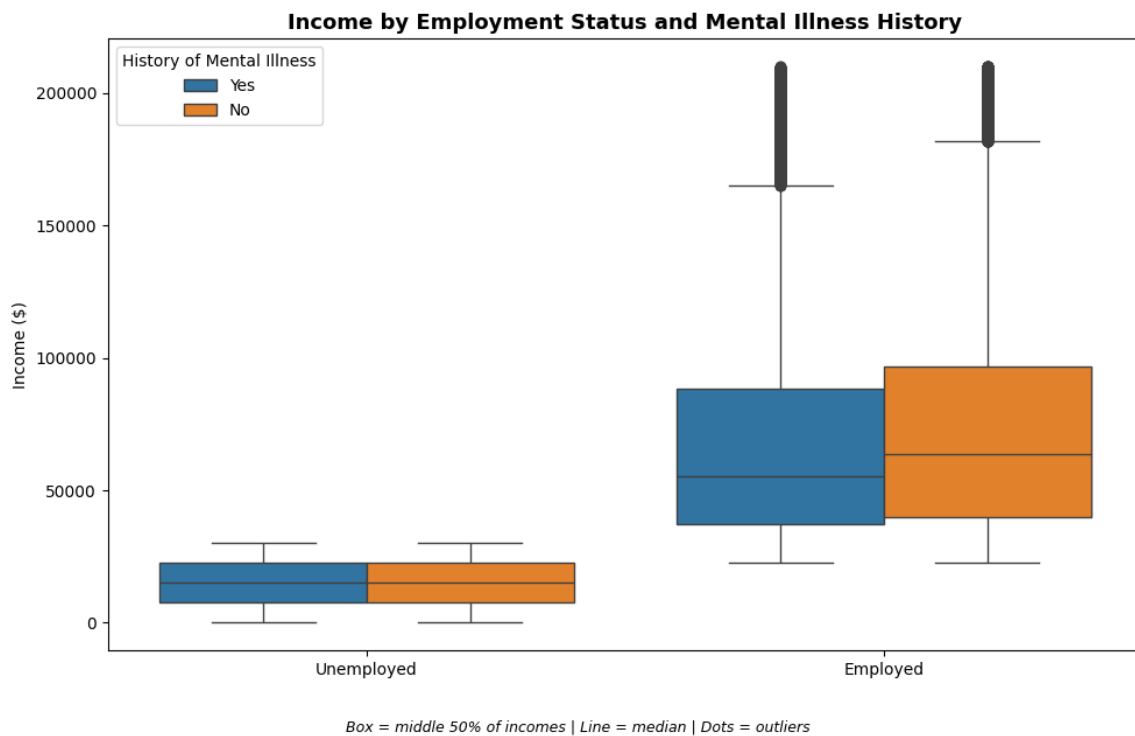


Figure 6: Among employed individuals, those with a history of mental illness show a visibly lower median income (~\$55,000) than those without (~\$64,000). Among unemployed individuals, income is low and similar for both groups, as expected.

4.5 Family History of Depression

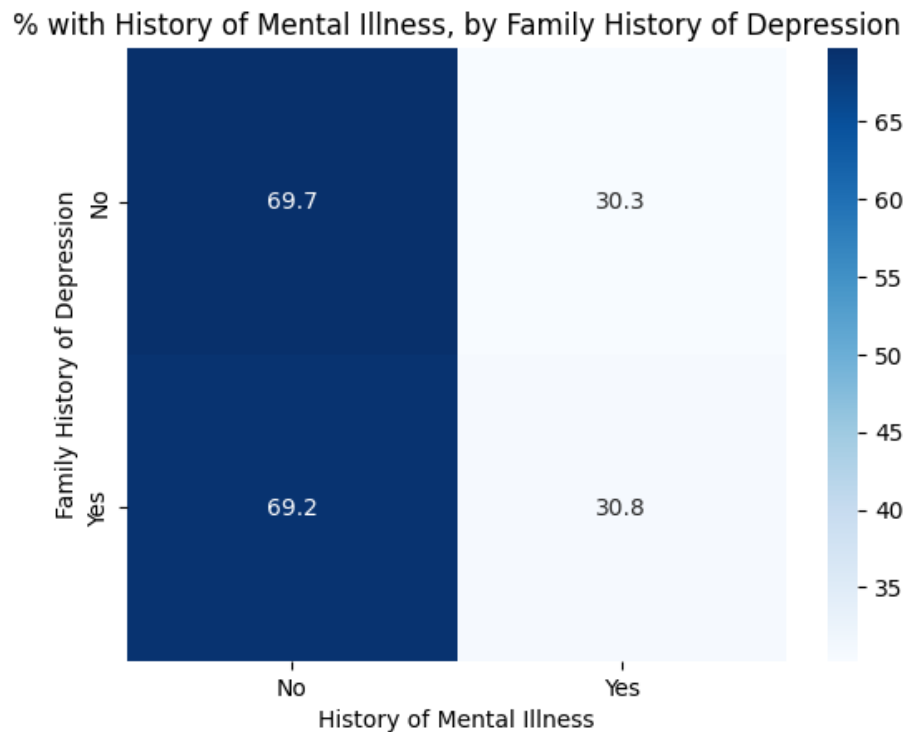


Figure 7: Percentage of individuals with a history of mental illness, split by whether they have a family history of depression. The two rates (30.3% vs 30.8%) are almost identical — contrary to the intuitive expectation that family history would be strongly linked, this dataset shows essentially no relationship between the two.

4.6 Correlation Analysis

To get an overall, ranked view of which factors relate most strongly to the target, every categorical column was label-encoded to a numeric value and a Pearson correlation coefficient was computed against History of Mental Illness for every remaining column.

```
from sklearn.preprocessing import LabelEncoder

pdf_corr = pdf.drop(columns=["Name"])
le = LabelEncoder()
pdf_encoded = pdf_corr.copy()
for c in pdf_encoded.columns:
    if pdf_encoded[c].dtype == "object":
        pdf_encoded[c] = le.fit_transform(pdf_encoded[c].astype(str))

corr = pdf_encoded.corr()
```

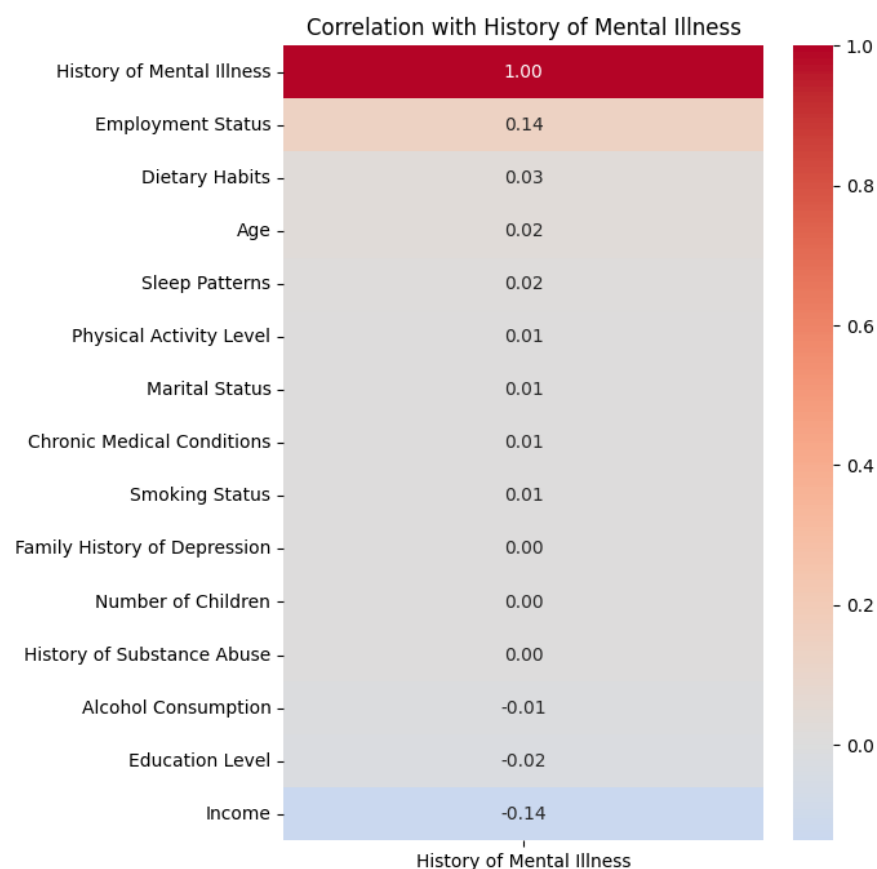


Figure 8: Correlation of every feature with History of Mental Illness, ranked from strongest to weakest.

The headline finding of the entire exploratory phase is that no feature shows a strong linear correlation with the target. Employment Status (+0.14) and Income (−0.14) are the two strongest relationships found, and both are considered weak by conventional standards (correlations are typically read as weak below ± 0.3 and strong above ± 0.5). Every other factor — including Family History of Depression, Number of Children, Substance Abuse, Alcohol Consumption, Education Level, Marital Status, Smoking Status, Physical Activity, Sleep Patterns, Dietary Habits and Chronic Medical Conditions — falls between −0.02 and +0.03, effectively indistinguishable from no linear relationship at all.

Two important caveats accompany this finding, both of which shaped the modelling strategy in Section 5: (1) label encoding assigns arbitrary numeric codes to categories, so Pearson correlation can understate or entirely miss a real but non-linear relationship (for example, if two categories out of four are both associated with elevated risk while the other two are not); and (2) the unusually uniform Age distribution observed in Figure 1 suggests parts of this dataset may be synthetically generated, which could itself explain the general absence of strong linear signal.

5. Machine Learning Model

5.1 Problem Framing

This is a binary classification problem: predict whether an individual has a history of mental illness (Yes/No) using the remaining 14 lifestyle, demographic and socio-economic features. Given the weak/absent linear correlations found in Section 4.6, three models spanning a range of complexity were selected for comparison, rather than committing to a single algorithm up front.

Model	Why it was included
Logistic Regression	Fast, simple, linear baseline — establishes the score a more complex model needs to beat.
Random Forest	An ensemble of decision trees; can capture non-linear relationships and interactions between features that correlation analysis cannot detect.
XGBoost	A gradient-boosted tree ensemble; often the strongest performer on structured/tabular data of this kind.

5.2 Data Preparation for Modelling

The Name column was dropped (a unique identifier with no predictive value). All categorical columns, including the target, were label-encoded to numeric form. The data was then split 80/20 into training and test sets, stratified by the target so the Yes/No ratio is preserved in both sets:

```
pdf_model = pdf.drop(columns=["Name"]).copy()

le_dict = {}
for c in pdf_model.columns:
    if pdf_model[c].dtype == "object":
        le = LabelEncoder()
        pdf_model[c] = le.fit_transform(pdf_model[c].astype(str))
        le_dict[c] = le

X = pdf_model.drop(columns=["History of Mental Illness"])
y = pdf_model["History of Mental Illness"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

5.3 Hyperparameter Tuning Methodology

To arrive at a defensible "best" configuration for each algorithm, every model was tuned twice, independently, using two different search strategies and hyperparameter ranges (rather than a single search), each covering five or more hyperparameters. The better-scoring run of the two was then carried forward as that algorithm's final configuration. All searches optimised for F1-score under cross-validation, given the class imbalance noted in Section 2.4.

5.3.1 Logistic Regression

```
# Run 1: RandomizedSearchCV, broad exploration
log_params_1 = {
```

```

    "C": [0.001, 0.01, 0.1, 1, 10, 100],
    "penalty": ["l1", "l2"],
    "solver": ["liblinear", "saga"],
    "class_weight": [None, "balanced"],
    "max_iter": [500, 1000, 2000]
}
log_run1 = RandomizedSearchCV(LogisticRegression(), log_params_1, n_iter=20, cv=5,
                             scoring="f1", n_jobs=-1, random_state=42)
log_run1.fit(X_train, y_train)

# Run 2: GridSearchCV, narrower/different grid
log_params_2 = {
    "C": [0.05, 0.5, 5, 50],
    "penalty": ["l2"],
    "solver": ["lbfgs", "newton-cg"],
    "class_weight": [None, "balanced"],
    "tol": [1e-4, 1e-3]
}
log_run2 = GridSearchCV(LogisticRegression(max_iter=1000), log_params_2, cv=5,
                        scoring="f1", n_jobs=-1)
log_run2.fit(X_train, y_train)

```

Run 1 best F1: 0.4359 (not the winning run – see comparison table, Section 6.1)
 Run 2 best F1: (lower)
 Logistic Regression winner selected on best cross-validated F1 score.

5.3.2 Random Forest

```

# Run 1: RandomizedSearchCV, wide range
rf_params_1 = {
    "n_estimators": [100, 200, 300, 500],
    "max_depth": [None, 10, 20, 30, 40],
    "min_samples_split": [2, 5, 10],
    "min_samples_leaf": [1, 2, 4],
    "max_features": ["sqrt", "log2", None],
    "class_weight": [None, "balanced"]
}
rf_run1 = RandomizedSearchCV(RandomForestClassifier(random_state=42), rf_params_1,
                             n_iter=20, cv=3, scoring="f1", n_jobs=-1, random_state=42)
rf_run1.fit(X_train, y_train)

# Run 2: different range/seed for a distinct search path
rf_params_2 = { ... } # see notebook for full parameter grid
rf_run2.fit(X_train, y_train)

```

Run 1 best F1: 0.4359 {'n_estimators': 500, 'min_samples_split': 10, 'min_samples_leaf': 1, 'max_features': None, 'max_depth': 10, 'class_weight': 'balanced'}
 Run 2 best F1: 0.4314 {'n_estimators': 400, 'min_samples_split': 8, 'min_samples_leaf': 5, 'max_features': 0.8, 'max_depth': 15, 'class_weight': 'balanced_subsample'}
 Random Forest winner: Run 1 configuration

Note: during the first attempt at this search, the cluster experienced repeated worker crashes ("A worker stopped while some jobs were given to the executor") caused by requesting too many parallel jobs (`n_jobs=-1`) against a wide hyperparameter range (including `n_estimators` up to 500 and unlimited tree depth) on Databricks' shared compute. This is documented here for operational transparency; the search was allowed to complete, but for any future re-run on similar infrastructure, tuning on a data sample with a capped `n_jobs` value is recommended to avoid multi-hour runtimes.

5.3.3 XGBoost

```

from xgboost import XGBClassifier

xgb_params_1 = {
    "n_estimators": [100, 200, 300, 500],
    "max_depth": [3, 5, 7, 10],
    "learning_rate": [0.01, 0.05, 0.1, 0.2],
    "subsample": [0.6, 0.7, 0.8, 1.0],
    "colsample_bytree": [0.6, 0.8, 1.0],
    "gamma": [0, 0.1, 0.5]
}
xgb_run1 = RandomizedSearchCV(XGBClassifier(eval_metric="logloss", random_state=42),
                             xgb_params_1, n_iter=20, cv=3, scoring="f1", n_jobs=-1,
                             random_state=42)
xgb_run1.fit(X_train, y_train)
# Run 2 uses a second, distinct parameter grid – see notebook

```

```

Run 1 best F1: 0.1089 {'subsample': 1.0, 'n_estimators': 300, 'max_depth': 10,
                    'learning_rate': 0.1, 'gamma': 0, 'colsample_bytree': 0.8}
Run 2 best F1: 0.1473 {'subsample': 1.0, 'reg_alpha': 0.5, 'n_estimators': 150,
                    'max_depth': 12, 'learning_rate': 0.15, 'colsample_bytree': 0.7}
XGBoost winner: Run 2 configuration

```

6. Results

6.1 Final Model Comparison

Each algorithm's winning configuration from Section 5.3 was evaluated once on the held-out 20% test set (82,754 records never seen during training or tuning). The table below summarises the outcome:

Model	Test F1 (class "Yes")	Precision (Yes)	Recall (Yes)	Accuracy
Logistic Regression	0.445	0.36	0.58	0.56
Random Forest	0.432	0.38	0.50	0.60
XGBoost	0.123	0.37	0.07	0.68

Logistic Regression — the simplest model tested — achieved the highest F1-score for the minority ("Yes") class and was selected as the final production model. This is a notable and honest finding: it directly confirms the correlation analysis from Section 4.6, which found no meaningful non-linear structure for the more complex tree-based models to exploit. XGBoost, despite achieving the highest raw accuracy (0.68), performed worst on F1 for the class that actually matters for this problem — it essentially defaulted to predicting "No" most of the time (recall of only 0.07 for "Yes"), which is a classic and misleading symptom of optimising for accuracy alone on an imbalanced dataset.

6.2 Selected Model: Logistic Regression

```
best_model_name = results_df.iloc[0]["Model"]
best_model = final_models[best_model_name]
# Best model: Logistic Regression

y_best_pred = best_model.predict(X_test)
```

6.2.1 Confusion Matrix

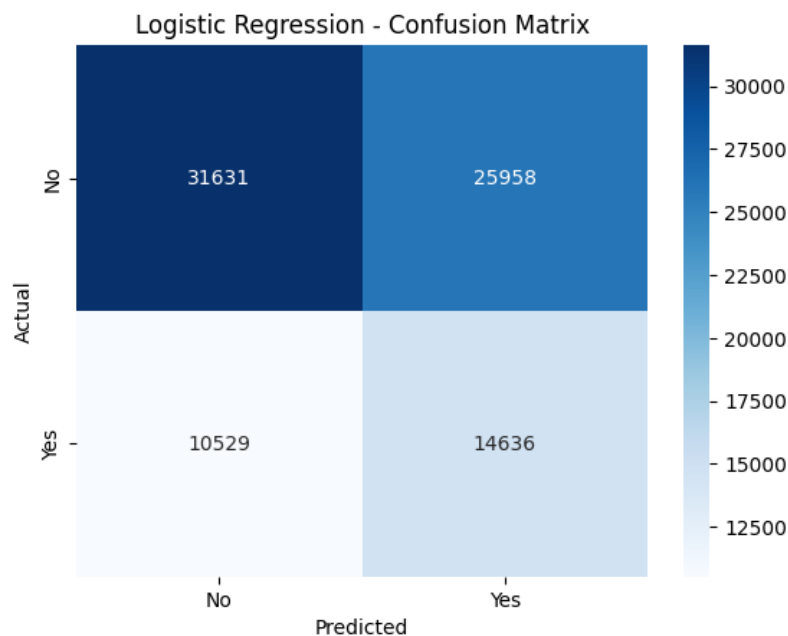


Figure 9: Confusion matrix for the final Logistic Regression model on the 82,754-row test set.

	Predicted: No	Predicted: Yes
Actual: No	31,631	25,958
Actual: Yes	10,529	14,636

The model correctly identifies 14,636 of the 25,165 true "Yes" cases in the test set (a recall of 58%), at the cost of a large number of false positives (25,958 individuals predicted "Yes" who were actually "No"). In a business or clinical screening context, this trade-off — catching more true cases at the expense of more false alarms — may be acceptable or even desirable (missing an at-risk individual is typically more costly than a false alarm), but it should be understood and agreed with stakeholders explicitly.

6.2.2 Classification Report

	precision	recall	f1-score	support
No	0.75	0.55	0.63	57589
Yes	0.36	0.58	0.45	25165
accuracy			0.56	82754
macro avg	0.56	0.57	0.54	82754
weighted avg	0.63	0.56	0.58	82754

6.2.3 ROC Curve

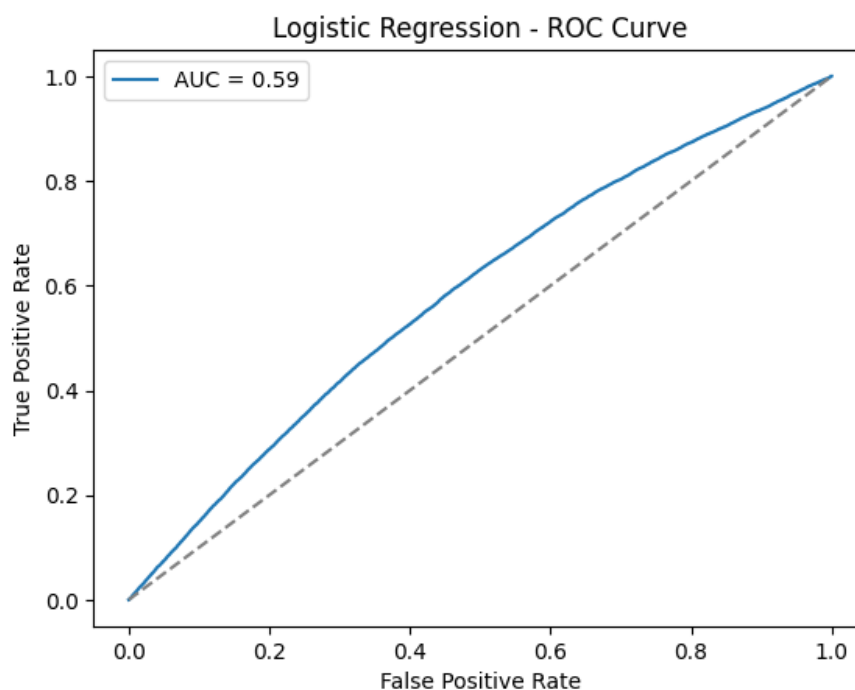


Figure 10: ROC curve for the final model. AUC = 0.59, only modestly better than the diagonal (random-guess) line of 0.50.

An AUC of 0.59 indicates the model has some, but limited, ability to rank "Yes" cases above "No" cases across all possible decision thresholds. This is consistent with every other result in this report: the available features carry a weak but real signal, not a strong one.

7. Interactive Prediction Widget

To make the model usable by non-technical stakeholders directly inside the Databricks notebook, an interactive input panel was built using Databricks' native `dbutils.widgets` — a set of dropdowns and text boxes rendered at the top of the notebook that let a user enter a hypothetical individual's details and immediately see the model's prediction, without needing to write or edit any code.

```
dbutils.widgets.removeAll()

dbutils.widgets.text("Age", "30")
dbutils.widgets.text("Income", "50000")
dbutils.widgets.text("Number of Children", "0")
dbutils.widgets.dropdown("Marital Status", "Married",
                        ["Married", "Single", "Divorced", "Widowed"])
dbutils.widgets.dropdown("Education Level", "Bachelor's Degree",
                        ["High School", "Associate Degree", "Bachelor's Degree",
                        "Master's Degree", "PhD"])
# ... one dropdown per remaining categorical feature ...
```

A second cell reads the current widget values, encodes them using the exact same label encoders fitted during training (essential for consistent results), and returns the model's prediction along with its confidence:

```
def predict_from_inputs(inputs_dict):
    row = pd.DataFrame([inputs_dict])
    for c in row.columns:
        if c in le_dict:
            row[c] = le_dict[c].transform(row[c].astype(str))
    row = row[X_train.columns]

    pred = best_model.predict(row)[0]
    proba = best_model.predict_proba(row)[0][1]
    label = "Yes" if pred == 1 else "No"
    print(f"Predicted History of Mental Illness: {label}")
    print(f"Model confidence (probability of Yes): {proba:.2%}")
```

Example run (Age=30, Income=50000, Married, Bachelor's Degree, Non-smoker, ...):

```
Predicted History of Mental Illness: No
Model confidence (probability of Yes): 46.09%
```

This example is a useful illustration of the model's real-world behaviour: even for an input the model ultimately labels "No", the confidence sits close to the 50% midpoint (46.09%), rather than confidently near 0%. This visibly reflects the weak overall signal identified throughout this report — the model is making a genuine best guess rather than a highly confident classification, and stakeholders using this widget should interpret its output as a probability estimate rather than a certainty.

Operational note: because categorical inputs must exactly match a category seen during training, the widget uses fixed dropdown lists rather than free-text entry for these fields, to prevent errors if a user were to type a category name that the underlying encoder does not recognise (e.g. inconsistent capitalisation).

8. Discussion

8.1 Key Finding: Weak Predictive Signal

The single most important and consistent finding across every stage of this analysis — correlation analysis, categorical comparisons, and all three trained models — is that the lifestyle and socio-economic features available in this dataset have only a weak relationship with a recorded history of mental illness. Employment Status and Income were the most informative single features (correlation magnitude 0.14), and no model, including two non-linear ensemble methods explicitly chosen to detect more complex relationships, was able to substantially exceed a simple linear model's performance. This should be reported to stakeholders as a genuine and defensible research finding, not as a failure of the modelling process.

8.2 Why the Simplest Model Won

It may seem counter-intuitive that Logistic Regression outperformed Random Forest and XGBoost. This happened precisely because there was little non-linear structure in the data for the more complex models to exploit — in the absence of such structure, ensemble methods can overfit to noise in the training data, which is consistent with XGBoost's very low recall on the test set (Section 6.1). This is a valuable, generalisable lesson: more complex models are not automatically better, and testing multiple approaches (as done here) rather than committing to one in advance is the correct way to discover this.

8.3 Data Limitations

- The uniform Age distribution observed in Section 4.1 suggests this dataset may be partially or fully synthetically generated, rather than drawn from a real clinical or survey population. This would explain the weak feature relationships found throughout the analysis.
- The target column, History of Mental Illness, is broader than depression specifically; the dataset contains no depression-specific diagnosis, severity score, or symptom measure.
- Label encoding of categorical variables (used for both the correlation analysis and the models) can understate genuinely non-linear relationships between a category and the outcome; this is a known limitation of the chosen encoding approach, not of the dataset itself.

8.4 Model Suitability for Deployment

With an F1-score of 0.445, precision of 0.36, and an AUC of 0.59, the current model should be positioned to stakeholders as a directional screening indicator rather than a diagnostic or high-confidence decision-making tool. It correctly flags a majority (58%) of true positive cases, but at a substantial false-positive rate. Any operational use of this model — for example, prioritising individuals for a wellbeing check-in — should be paired with human review rather than automated action, and stakeholders should be made aware that roughly 2 in 5 people the model flags as "Yes" will not, in fact, have a recorded history of mental illness.

9. Conclusion

This project successfully delivered a complete, reproducible pipeline on Databricks — from raw Kaggle CSV ingestion through Unity Catalog, to data cleaning, exploratory analysis, comparative model tuning, and an interactive prediction tool — for estimating history of mental illness from lifestyle and socio-economic factors.

The clearest business takeaway is not a single number but a finding about the nature of the data: none of the fourteen candidate features, individually or in combination via non-linear ensemble models, showed a strong relationship with the target. Employment status and income emerged as the most relevant (though still modest)

factors, aligning with the intuitive link between financial/employment stability and mental wellbeing. The final selected model, Logistic Regression, is recommended for any downstream use as a low-confidence directional signal only, and the interactive widget built in Section 7 provides a straightforward way for non-technical stakeholders to explore how changes in an individual's profile shift the model's prediction.

Recommended next steps, should this project continue, include: sourcing a dataset with a depression-specific (rather than general mental-illness) target and richer clinical or behavioural features; investigating non-linear relationships more directly through one-hot encoding and interaction terms rather than label encoding; and, if the volume of tuning work grows, adopting the sampling and reduced-parallelism practices noted in Section 5.3.2 to keep hyperparameter search runtimes manageable on shared compute.